

Mercari Price Suggestion Challenge

텍스트·범주형·수치형 피처를 결합한 **XGBoost** 회귀 파이프라인 분석
— 데이터 전처리부터 HyperOpt 튜닝, RMSLE 성능 검증까지

RMSLE 회귀 분석

NLP · TF-IDF

XGBoost + HyperOpt

Ridge 베이스라인

Feature Engineering

1,482,535

학습 데이터 행(row) 수

8

원본 컬럼 수

105,812

최종 피처 행렬 차원

RMSLE

핵심 성능 지표

01 서론

Introduction · 문제 배경과 보고서의 범위

Mercari는 미국의 대표적인 C2C(개인 간) 중고거래 플랫폼이다. 이런 플랫폼에서 판매자가 가장 먼저 마주치는 어려움은 “이 물건을 얼마에 올려야 하는가”라는 질문이다. 같은 브랜드의 같은 종류 상품이라도 상태, 카테고리, 설명을 얼마나 성의 있게 썼는지에 따라 팔리는 가격이 크게 달라진다. 사람이 일일이 비슷한 상품을 검색해서 시세를 비교하는 것은 비효율적이므로, Mercari는 2018년 Kaggle에 “판매자가 상품 정보를 입력하면 적정 가격을 자동으로 제안해 주는 모델”을 만드는 경진대회를 열었다. 이것이 **Mercari Price Suggestion Challenge**이다.

본 보고서는 이 경진대회 데이터셋을 이용해서, 상품의 텍스트 정보(이름, 설명)와 범주형 정보(카테고리, 브랜드, 상태, 배송 조건)만으로 판매 가격을 예측하는 회귀 모델을 처음부터 구축하는 과정을 정리한다. 데이터 탐색(EDA) → 전처리 → 피쳐 엔지니어링 → 모델 학습·평가까지 전체 파이프라인을 다루며, 특히 **선형 베이스라인(Ridge)**과 **트리 부스팅 모델(XGBoost)**이 **초고차원 희소 텍스트 데이터에서 어떻게 다르게 동작하는지**를 비교하는 데 중점을 두었다. XGBoost에는 HyperOpt 기반 하이퍼파라미터 자동 탐색까지 적용했다.

참고 자료 · Mercari Price Suggestion Challenge – A Deep Learning Regression Case Study (Medium, Shiv Shankar Dutta)

관련 코드 · 04 Mercari price suggestion/Mercari price suggestion XGB.ipynb (EDA 시각화는 03_회귀/results/figures/ 산출물 재사용)

02 문제 제기

무엇을, 왜 어렵게 예측하는가

2.1 무엇을 예측하는가

구분	내용
입력(피처)	<code>name</code> (상품명), <code>category_name</code> (카테고리), <code>brand_name</code> (브랜드), <code>item_condition_id</code> (상태), <code>shipping</code> (배송비 부담 주체), <code>item_description</code> (상세설명)
출력(타겟)	판매 가격(<code>price</code> , 단위 USD) — 학습 시 <code>log1p</code> 변환하여 사용
문제 유형	지도학습 회귀(Regression) — 예측값이 카테고리가 아니라 연속적인 숫자값이라는 점에서 분류(Classification)와 구분된다

2.2 왜 어려운 문제인가

- **대용량 데이터** — 학습 데이터만 약 148만 건이다. 작은 데이터셋에서 잘 통하는 방법(예: 모든 피처 조합을 다항식으로 확장하는 것)이 그대로 통하지 않는다.
- **결측치** — `brand_name` 은 약 42.7%가 비어 있다(자세한 수치는 5.1절). 브랜드를 안 적은 상품과 실제로 브랜드가 없는 상품(수공예품 등)을 구분해야 한다.
- **비정형 텍스트 → 정형 피처 변환** — 머신러닝 모델은 숫자 행렬만 입력으로 받을 수 있다. `name`, `item_description` 같은 자유 서술형 텍스트를 어떻게 숫자로 바꿀지가 이 프로젝트의 핵심 난제이며, 그 결과로 수만~십만 개의 열(컬럼)을 가진 초고차원 데이터가 만들어진다(6.2절).
- **가격 분포의 비대칭성** — 가격은 대부분 몇 달러~몇십 달러에 몰려 있지만 극히 일부 고가 상품이 꼬리처럼 길게 늘어져 있다(오른쪽으로 치우친 분포, right-skewed). 이 상태로 그대로 학습하면 모델이 소수의 고가 상품 오차를 줄이는 데만 매달려 전반적인 예측력이 떨어진다(5.2절, 6절).

03 데이터셋 분석을 통한 모델 결정

피처의 성격이 모델 선택을 결정한다

3.1 피처 성격 파악

원본 데이터는 8개 컬럼(`train_id`, `name`, `item_condition_id`, `category_name`, `brand_name`, `shipping`, `item_description`, `price`)으로 구성되어 있다. 이 중 `train_id`는 단순 일련번호라 예측에 도움이 되지 않으므로 제외하면, 실질적으로 사용할 피처는 7개다. 이들은 성격이 크게 두 갈래로 나뉜다.

- **텍스트형:** `name` (짧은 상품명), `item_description` (긴 서술형 설명) — 자연어이므로 벡터화가 필요
- **범주형:** `category_name`, `brand_name`, `item_condition_id`, `shipping` — 종류가 정해진 값이므로 원-핫 인코딩이 적합

3.2 모델 선택 논리

텍스트를 벡터화하면(6.2절) 컬럼 수가 수만~십만 개로 폭증하는데, 이런 데이터는 다음 두 가지 성질을 가진다.

- 행보다 열이 훨씬 많거나 비슷한 수준이고, 각 행에서 대부분의 값이 0인 **희소(sparse) 행렬**이다 (상품 하나의 설명 글에 전체 사전 단어 중 극히 일부만 등장하기 때문).
- 개별 피처(단어 하나하나)와 가격의 관계가 대체로 **선형에 가깝다**(특정 단어가 있으면 가격이 대략 얼마쯤 오르내리는 식).

이런 데이터에서는 규제(Regularization)가 적용된 선형 회귀, 그중에서도 **릿지(Ridge) 회귀**가 정석적인 베이스라인으로 꼽힌다(8절에서 원리를 자세히 설명한다). 한편 결정 트리를 순차적으로 결합하는 **그래디언트 부스팅 모델(XGBoost)**은 정형 데이터 경진대회에서 가장 널리 쓰이는 강력한 기법이므로, 이 초고차원 희소 데이터에서도 선형 모델을 능가할 수 있는지 비교해 볼 가치가 있다(9절).

따라서 본 보고서는 **Ridge 회귀**를 베이스라인으로 삼고 **XGBoost**를 비교 모델로 채택했으며, XGBoost에는 **HyperOpt** 기반 자동 하이퍼파라미터 탐색까지 적용해 튜닝 전/후를 함께 비교한다. 이 구성은 초고차원 희소 데이터에서 선형 모델과 트리 부스팅 모델의 상대적 강약을 뚜렷하게 대비시킨다.

04 평가 지표

왜 정확도가 아니라 RMSLE인가

4.1 왜 정확도가 아니라 오차 지표를 쓰는가

분류 문제에서는 “맞았는가/틀렸는가”를 셀 수 있어 정확도(Accuracy) 같은 지표가 가능하지만, 회귀는 예측값이 연속적인 숫자이기 때문에 “얼마나 틀렸는가”의 크기를 측정해야 한다. 가장 기본적인 방법은 실제값과 예측값의 차이(오차, 잔차)를 제곱해서 더한 뒤 평균을 내는 MSE(Mean Squared Error) 방식이다.

$$MSE = (1/n) * \sum (y_{actual} - y_{pred})^2$$

제곱을 하는 이유는 오차의 부호(+/-)가 서로 상쇄되어 실제보다 작게 나오는 것을 막기 위함이다. 다만 MSE는 단위가 “가격의 제곱(달러²)”이 되어 버려서 직관적으로 해석하기 어렵다. 그래서 실무에서는 MSE에 제곱근을 씌운 **RMSE(Root MSE)**를 더 많이 쓴다 — 단위가 다시 원래 가격(달러)으로 돌아오기 때문이다.

4.2 RMSE에서 RMSLE로

Mercari 경진대회는 RMSE 대신 **RMSLE(Root Mean Squared Logarithmic Error)**를 공식 지표로 쓴다.

$$RMSLE = \sqrt{(1/n) * \sum (\log(1+y_{pred}) - \log(1+y_{actual}))^2}$$

RMSE와 다른 점은 오차를 계산하기 전에 예측값·실제값 모두에 로그를 씌운다는 것뿐이다. 이 사소해 보이는 차이가 실제로는 큰 의미를 가진다.

상황	RMSE 관점	RMSLE 관점
\$10 상품을 \$15로 예측 (오차 \$5)	오차 5, 매우 부정확한 예측으로 취급	오차 = $\log(16) - \log(11) \approx 0.375$ — 가격이 50% 부풀려진 것으로 취급
\$1,000 상품을 \$1,005로 예측 (오차 \$5)	오차 5, 위 사례와 동일하게 취급	오차 = $\log(1006) - \log(1001) \approx 0.005$ — 0.5%밖에 안 틀린 것으로 취급

즉 RMSE는 “달러 몇 개 차이 나는가”라는 **절대오차**를 보지만, RMSLE는 “몇 % 차이 나는가”라는 **상대오차**를 본다. \$1 짜리 액세서리부터 \$2,000짜리 명품까지 가격대가 넓은 데이터에서는, 절대오차 기준으로는 고가 상품 몇 건의 오차가 지표 전체를 지배해 버리므로 상대오차 기준인 RMSLE가 훨씬 공정하다.

① 상대오차 반영

저가·고가 상품의 오차를 공평한 비율로 비교

② 과소예측에 더 큰 페널티

$\log(1+x)$ 는 x 가 0에 가까울수록 기울기가 가팔라져, 실제보다 낮게 예측하는 실수에 더 민감 — 판매자의 “싸게 등록해 손해” 상황을 더 경계

③ 이상치에 강건

로그를 씌우면 극단적으로 큰 값의 영향력이 줄어든다(5.2절)

구현 단순화 — 타깃을 미리 $\log_1p(\text{price})$ 로 변환해서 학습하면, 로그 스케일에서의 일반 RMSE 계산이 곧 RMSLE와 같아진다. 본 프로젝트에서도 이 방식을 사용했으며, 최종 RMSLE는 예측값을 $\exp m1$ 로 원 스케일(\$)로 되돌린 뒤 계산했다.

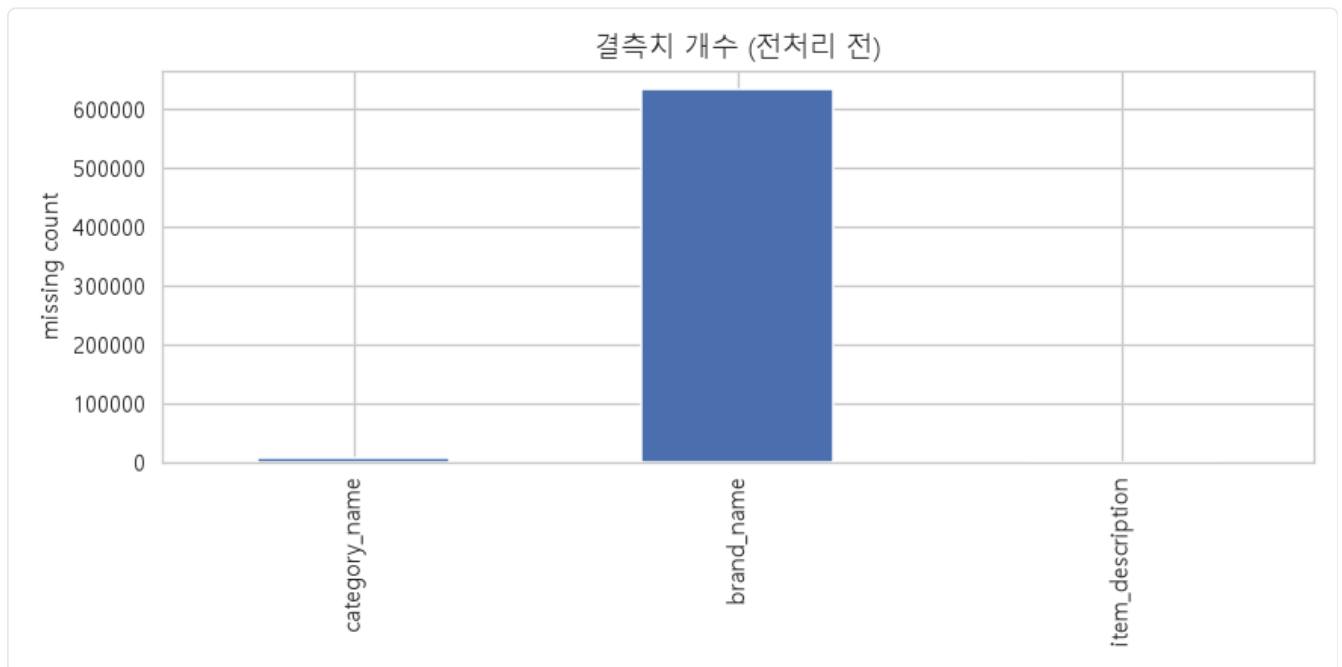
```
def rmsle(y, y_pred):  
    # log가 아닌 log1p로 계산해 underflow/overflow 방지  
    return np.sqrt(np.mean(np.power(np.log1p(y) - np.log1p(y_pred), 2)))  
  
def evaluate_org_price(y_test, preds):  
    # 타깃이 log1p 스케일이므로 expm1로 원복 후 RMSLE 산출  
    return rmsle(np.expm1(y_test), np.expm1(preds))
```

05 탐색적 데이터 분석 (EDA)

전체 1,482,535행 × 8열 · 모델링 전 데이터의 모양 확인

5.1 결측치 확인 (전처리 전)

컬럼	결측 개수	비고
<code>category_name</code>	6,327	전체의 약 0.4% (non-null 1,476,208)
<code>brand_name</code>	632,682	전체의 약 42.7% — 절반 가까이 브랜드가 비어 있음 (non-null 849,853)
<code>item_description</code>	6 + 82,489	NaN 6건 + 'No description yet' 문자열 82,489건. 판매자가 설명을 안 쓰면 사이트가 자동으로 placeholder 문자열을 채우므로, 겉보기엔 결측이 없어 보여도 실질적으로는 “설명 없음” 상태다

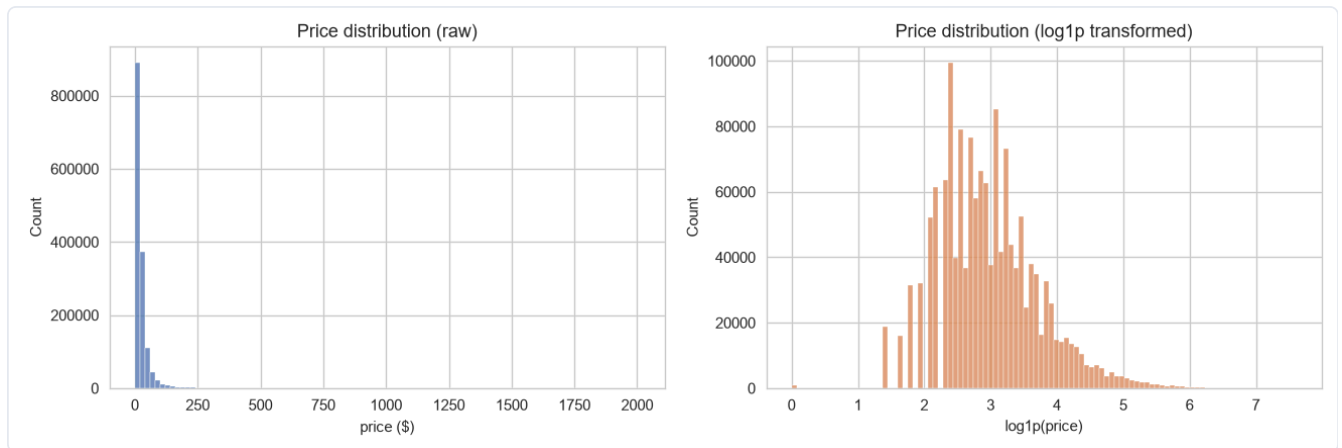


결측치 개수 — 전처리 전

해석: `brand_name` 결측이 42.7%나 되는 것은 데이터 수집상의 오류가 아니라, 실제로 브랜드가 없는 핸드메이드/무명 상품이 많다는 뜻일 가능성이 높다. 따라서 이 결측치를 버리지 않고 “브랜드 없음”이라는 하나의 값('Other_Null')으로 인코딩하는 것이 합리적이다(6.1절).

5.2 가격 분포 — 로그 변환 전/후 비교

구간	값
원본 가격의 왜도(skewness)	11.39
log1p 변환 후 왜도	0.66

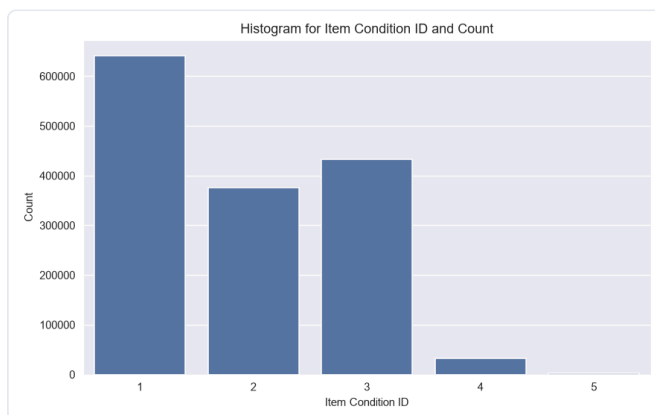


Price distribution — raw(좌) vs log1p(우)

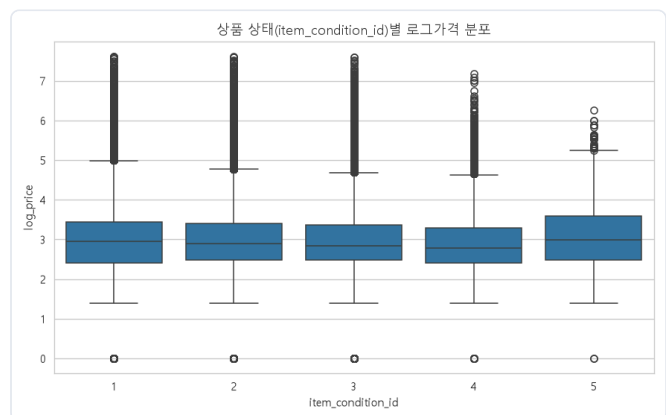
왜도(skewness)는 분포가 좌우 대칭에서 얼마나 벗어나 있는지를 나타내는 통계량이다. 0에 가까울수록 대칭(정규분포에 가까움), 값이 클수록 한쪽으로 길게 늘어진 꼬리가 있다는 뜻이다. 원본 왜도 11.39는 극단적으로 오른쪽 꼬리가 긴 분포이며, 대부분의 상품이 \$0~50 구간에 몰려 있고 \$500 이상 고가 상품은 거의 안 보일 정도로 드물다.

이런 분포를 그대로 학습에 쓰면, RSS/MSE 기반 손실 함수가 소수 고가 상품의 큰 절대오차를 줄이는 데 매달려 학습이 왜곡되고, 선형 모델의 안정성도 떨어진다. log1p 변환(정확히는 $\log(1+price)$) 후 왜도가 0.66으로 크게 낮아졌으며, 이는 4.2절 RMSLE 지표와도 정확히 맞아떨어지는 전처리다.

5.3 상품 상태(item_condition_id)별 가격



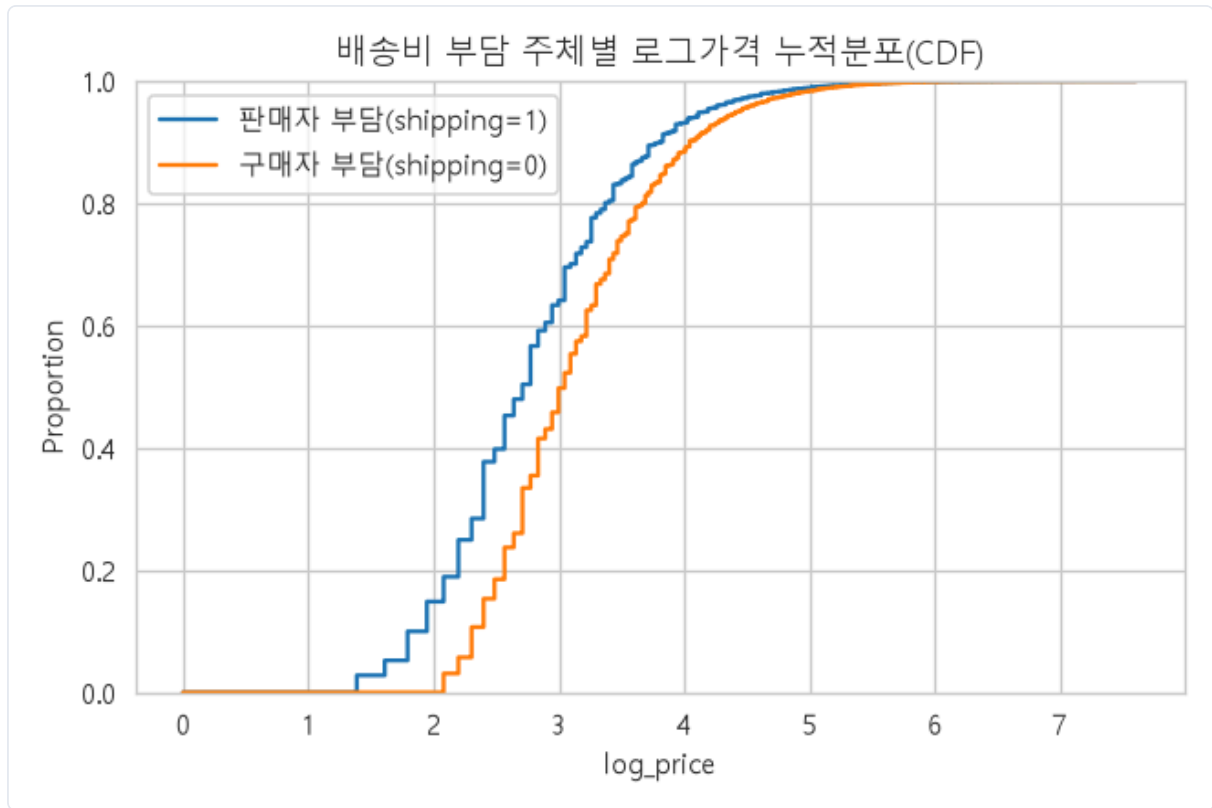
item_condition_id 분포 (1:640,549 · 2:375,479 · 3:432,161 · 4:31,962 · 5:2,384)



log(price) by item_condition_id

item_condition_id 는 1(새 상품)부터 5(불량)까지의 척도다. 상식적으로는 상태가 나쁠수록 가격이 낮아야 하지만, 상태에 따른 가격 차이가 뚜렷하지 않고 오히려 상태 5에서도 예외적으로 높은 가격대가 관측된다. 골동품·한정판처럼 “새것 여부”보다 “희소성”이 가격을 좌우하는 상품이 섞여 있기 때문으로 추정된다. 즉 이 피처 하나만으로는 가격을 설명하기 부족하며, 다른 피처(브랜드·카테고리·텍스트)와 함께 봐야 의미가 있다.

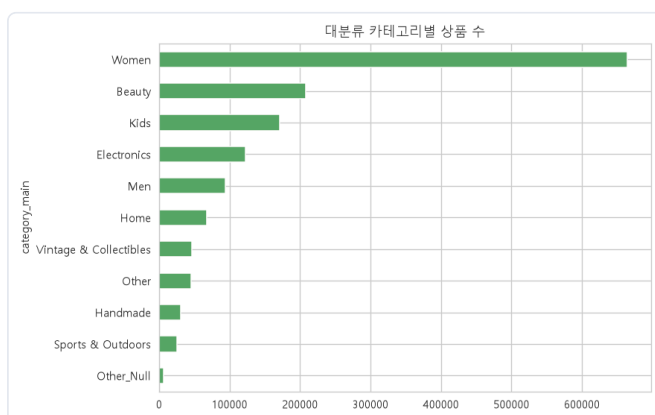
5.4 배송비 부담 주체(shipping)별 가격



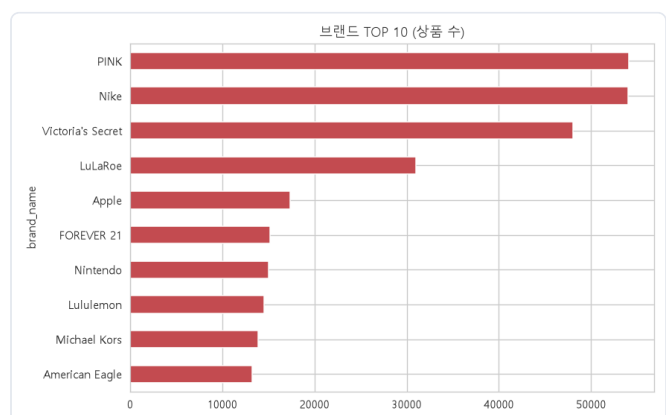
배송비 부담 주체별 로그가격 누적분포(CDF) · buyer 부담 819,435건 / seller 부담 663,100건

누적분포함수(CDF)에서 x축의 특정 가격 지점의 y축 값은 “그 가격 이하인 상품의 비율”을 뜻한다. 판매자가 배송비를 부담하는 경우 (`shipping`=1)의 곡선이 오른쪽에 위치한다 — 같은 누적 비율에 도달하려면 더 높은 가격이 필요하다는 뜻이므로, 전반적으로 가격이 더 높게 형성되어 있다. 판매자가 배송비를 상품 가격에 미리 반영하는 합리적인 가격 책정 행태로 해석된다.

5.5 카테고리 / 브랜드 분포



대분류(cat_dae)별 상품 수



브랜드 TOP 10 (상품 수)

`category_name` 은 슬래시(/)로 구분된 대/중/소 계층 구조 문자열이다(예: Women/Tops & Blouses/T-Shirts). 이를 그대로 벡터화하면 계층 정보를 활용하기 어려우므로, 슬래시 기준으로 3개 컬럼(`cat_dae`, `cat_jung`, `cat_so`)으로 분리했다(6.1절). 대분류는 11종, 중분류 114종, 소분류 871종이다. 대분류 기준으로는 Women(여성 의류)이 664,385건으로 압도적이며(Beauty 207,828 · Kids 171,689 · Electronics 122,690 ...), 모델이 “여성 의류”에 과적합되지 않도록 주의해야

한다. 브랜드는 전체 고유값이 4,810개지만 상위 소수(Other_Null 632,682 · PINK 54,088 · Nike 54,043 · Victoria's Secret 48,036 · LuLaRoe 31,024)가 거래의 큰 부분을 차지하는 롱테일 분포로, 원-핫 인코딩 시 대부분의 브랜드 열이 극히 드물게만 1이 되는 희소 행렬을 만든다(6.2절과 직결).

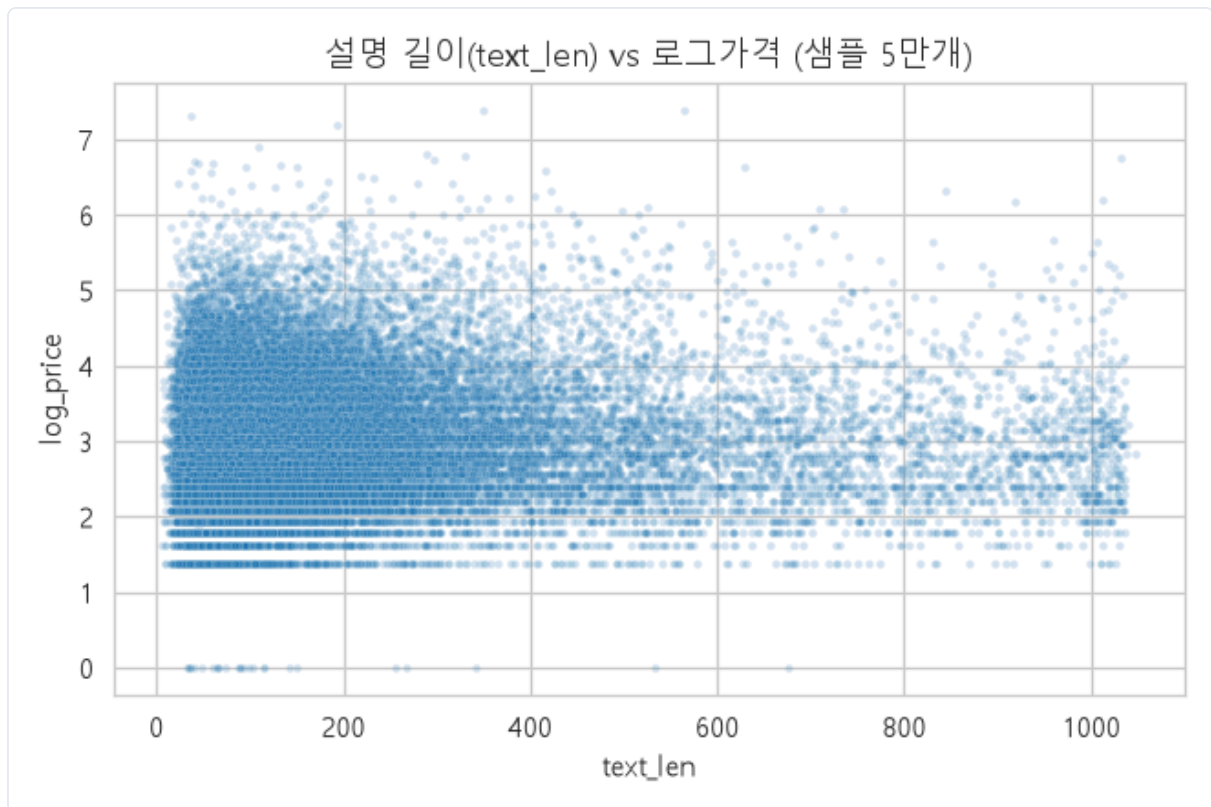
5.6 상품명 워드클라우드



WordCloud — name

PINK, Nike, Apple, iPhone, Victoria's Secret 같은 브랜드명과 free shipping, bundle, new 같은 거래 관련 표현이 두드러진다. 상품명(name) 텍스트 자체에 이미 브랜드·상태 정보가 자연어로 녹아 있다는 뜻이며, 6.2절에서 name을 벡터화할 때 이런 반복 단어들이 유의미한 피처가 된다.

5.7 텍스트 길이와 가격의 관계



text length vs log(price) (sample 50k)

`name` + `item_description` 을 합친 전체 텍스트 길이와 로그 가격의 산점도다 (`item_description` 평균 문자열 길이는 145.7자). 뚜렷한 선형 상관관계는 보이지 않지만, 매우 짧은 텍스트(설명을 거의 안 쓴 경우)가 저가 구간에 몰리는 경향이 약하게 관찰된다. 상관계수가 강하지 않으므로 단독 길이 피쳐보다 6.2절의 텍스트 벡터화(단어 자체의 의미)가 더 중요한 정보원이 된다.

06 데이터 전처리 및 특징 추출

결측치 처리 → 계층 분할 → 벡터화 → 희소 행렬 결합

6.1 결측치 처리 (전처리 후) — 전/후 비교 완성

5.1절에서 확인한 결측치를 다음 규칙으로 채운다.

- `brand_name`, `category_name` 결측 → 'Other_Null' ("없음" 자체를 하나의 유효한 범주로 취급)
- `item_description` 결측(NaN) → 'Other_Null'
- `category_name` 은 `split_cat()`으로 대/중/소 3개 컬럼 (`cat_dae` / `cat_jung` / `cat_so`)으로 분리하며, 분할 불가능한 결측은 세 컬럼 모두 'Other_Null'로 예외 처리

```
def split_cat(category_name):
    try:
        return category_name.split('/')
    except:
        return ['Other_Null', 'Other_Null', 'Other_Null'] # 결측치 예외 처리

mercari_df['cat_dae'], mercari_df['cat_jung'], mercari_df['cat_so'] = \
    zip(*mercari_df['category_name'].apply(lambda x: split_cat(x)))
```

'Other_Null'처럼 결측을 별도 문자열로 바꾸는 방식은, 뒤에서 `LabelBinarizer`로 원-핫 인코딩할 때 "브랜드 없음"도 다른 브랜드들과 동등한 하나의 범주(열)로 자동 포함되게 하기 위함이다. 결측을 그냥 버리거나 최빈값으로 채우면 "브랜드가 없다"는 중요한 정보 자체가 사라진다. 처리 후 대상 컬럼의 결측치는 모두 0으로 확인되었다.



Missing value count — 전처리 후 (5.1절 그래프와 비교)

6.2 피처 벡터화 — 텍스트를 숫자로 바꾸기

머신러닝 모델은 문자열을 직접 이해하지 못하므로 텍스트를 숫자 벡터로 바꾸는 **피처 벡터화**가 필요하다. 사이킷런의 CountVectorizer·TfidfVectorizer는 모두 BOW(Bag of Words) 방식으로, 단어 순서를 무시하고 “어떤 단어가 몇 번 등장했는가”로 문서를 표현한다. TfidfVectorizer는 여기에 더해 여러 문서에 공통으로 흔한 단어(“the”, “good” 등)의 가중치를 낮추는 IDF를 곱한다. 본 프로젝트에서는 **짧은 상품명도 TF-IDF로 통일 처리**하되 n-gram 범위를 (1,2)로 좁게, 서술형 설명은 (1,3)으로 넓게 잡았다.

피처	방법	설정	결과 차원
<code>name</code>	TfidfVectorizer	ngram_range=(1,2), max_features=50000, stop_words='english'	50,000
<code>item_description</code>	TfidfVectorizer	ngram_range=(1,3), max_features=50000, stop_words='english' — “free shipping”처럼 붙어야 의미가 통하는 표현을 포착	50,000
<code>brand_name</code>	LabelBinarizer	sparse_output=True	4,810
<code>item_condition_id</code>	LabelBinarizer	sparse_output=True	5
<code>shipping</code>	LabelBinarizer	sparse_output=True — 이진값이라 1차원으로 축소	1
<code>cat_dae</code>	LabelBinarizer	sparse_output=True	11
<code>cat_jung</code>	LabelBinarizer	sparse_output=True	114
<code>cat_so</code>	LabelBinarizer	sparse_output=True	871

```
from scipy.sparse import hstack

sparse_matrix_list = (X_descp, X_name, X_brand, X_item_cond_id,
                      X_shipping, X_cat_dae, X_cat_jung, X_cat_so)
X_features_sparse = hstack(sparse_matrix_list).tocsr()
# -> (1482535, 105812)
```

왜 희소 행렬(sparse matrix)인가 — `name` + `item_description` 만 10만 개의 열이 생기는데, 상품 하나에서 값이 0이 아닌 칸은 극소수다. CSR(Compressed Sparse Row) 형식으로 저장하면 148만 × 10.6만 행렬도 실제 메모리는 0이 아닌 값의 개수에 비례한다. 일반 배열(dense)로 저장하면 1,482,535 × 105,812 ≈ **1,569억 칸**을 모두 올려야 해서 현실적으로 불가능하다.

위 8개 피처를 `scipy.sparse.hstack`(가로 결합)으로 이어 붙여 최종 피처 행렬 `X_features_sparse`를 구성했다. **최종 shape: (1,482,535, 105,812)** (`name` 50,000 + `item_description` 50,000 + `brand_name` 4,810 + `item_condition_id` 5 + `shipping` 1 + `cat_dae` 11 + `cat_jung` 114 + `cat_so` 871).

07 기존 솔루션 검토

참고 아티클(Shiv Shankar Dutta)의 모델별 RMSLE

모델	기법	RMSLE	비고
Ridge	TF-IDF + One-Hot	0.444	학습 2분, 빠르고 안정적
XGBoost	TF-IDF + One-Hot	0.45	학습 2시간, Ridge보다 낮은 성능
LightGBM	TF-IDF + One-Hot	0.439	학습 1시간, Ridge 대비 소폭 개선

트리 기반 부스팅 모델(XGBoost/LightGBM)은 텍스트 벡터화로 인한 초고차원 희소 행렬에서도 선형 모델(Ridge) 대비 뚜렷한 이점을 보이지 못했다는 것이 아티클의 핵심 결론이다. 특히 XGBoost는 학습 시간이 Ridge의 수십 배에 달하더라도 성능은 오히려 뒤처졌다. 이 결론은 본 보고서 10절의 실측 결과와도 같은 방향으로 재확인된다.

08 기준 모델 (Ridge)

규제가 적용된 선형 회귀 — 초고차원 희소 데이터의 정석 베이스라인

8.1 선형 회귀의 기본 원리

회귀의 목표는 실제값과 예측값의 차이(오차)를 최소화하는 회귀 계수(가중치)를 찾는 것이다. 가장 단순한 단순 선형 회귀는 $y_{\text{pred}} = w_0 + w_1 * x$ 형태의 직선으로 데이터를 근사하며, 이 직선과 실제 데이터 사이의 오차 제곱합을 RSS(Residual Sum of Squares)라 부른다.

$$\text{RSS}(w_0, w_1) = (1/N) * \sum (y_{\text{actual}} - (w_0 + w_1 * x))^2$$

회귀 모델 학습은 이 RSS(비용 함수)를 최소로 만드는 w 값을 찾는 최적화 문제다. Mercari 데이터처럼 피처가 10만 개가 넘으면 w 도 10만 개가 되는데, 이렇게 피처 수가 매우 많을 때 그냥 RSS만 최소화하면 (1) 과적합, (2) 서로 비슷한 정보를 담은 피처(예: "free"와 "shipping"이 항상 같이 등장)가 있을 때 계수가 불안정하게 널뛰는 문제가 생긴다.

8.2 릿지(Ridge) 회귀 — 규제(Regularization)

이 문제를 해결하기 위해 비용 함수에 "회귀 계수가 너무 커지지 않도록 벌점을 매기는" 항을 추가한 것이 **규제(Regularization)**다. 릿지 회귀는 회귀 계수의 제곱합(L2 규제)을 비용 함수에 더한다.

$$\text{Cost}(w) = \text{RSS}(w) + \alpha * \sum w_j^2$$

α 는 규제 강도를 조절하는 하이퍼파라미터로, 클수록 계수를 0에 가깝게 눌러 모델을 단순하게 만든다. 릿지는 계수를 0에 가깝게 줄일 뿐 완전히 0으로 만들지는 않는다는 점에서, 피처를 아예 제거하는 라쏘(Lasso, L1)와 구분된다. Mercari처럼 10만 개 피처 각각이 (드물게라도) 어느 정도 정보를 담고 있을 수 있는 상황에서는, 모든 피처의 영향력을 완만하게 줄이는 릿지가 더 안정적인 선택이다.

8.3 학습 설정 및 결과

```
linear_model = Ridge(solver='lsqr', fit_intercept=False, alpha=5)
# train/test split: test_size=0.2, random_state=156
```

피처 구성	RMSLE
item_description 제외	0.48561
item_description 포함 (최종)	0.45762

`fit_intercept=False`로 설정한 이유는, 원-핫 인코딩된 범주형 피처들(브랜드·카테고리 등)이 이미 각 범주별로 하나씩 열을 가지고 있어 사실상 절편의 역할을 대신할 수 있기 때문이다. `solver='lsqr'`은 희소 행렬처럼 매우 큰 데이터에 최적화된 최소제곱법 알고리즘이다. `item_description`을 피처에 포함하자 RMSLE가 0.48561 → 0.45762로 개선되어, 상세 설명 텍스트가 가격 예측에 유의미한 신호를 담고 있음을 확인할 수 있다.

09 최종 모델 (XGBoost)

그래디언트 부스팅 + HyperOpt 자동 튜닝

9.1 앙상블과 부스팅의 원리

여러 개의 모델을 결합해 예측 성능을 높이는 방법을 **앙상블(Ensemble)**이라 하며, 크게 배깅(Bagging)과 부스팅(Boosting)으로 나뉜다.

- **배깅**(대표: 랜덤 포레스트): 여러 결정 트리를 서로 독립적으로 학습시킨 뒤 결과를 평균/투표로 합친다.
- **부스팅**(대표: GBM, XGBoost, LightGBM): 결정 트리를 하나씩 순차적으로 학습시키되, 이전 트리가 잘못 예측한 부분(오차)을 다음 트리가 집중적으로 보완하도록 학습한다. 트리 하나하나의 “약한 학습기(weak learner)”이지만, 오차를 계속 보완해 나가면서 전체적으로는 강력한 모델이 된다.

XGBoost는 그래디언트 부스팅 계열 중에서도 (1) 손실 함수를 2차(Newton)까지 근사해 분할 이득을 정확히 계산하고, (2) 리프 가중치에 L1·L2 규제를 걸어 과적합을 억제하며, (3) 연속형 값을 구간(히스토그램, `tree_method='hist'`, `max_bin`)으로 미리 나눠 분할 탐색 속도를 높인 구현체다. 정형 데이터 경진대회에서 오랫동안 표준으로 쓰여 왔다.

9.2 학습 설정 및 결과

① 기본 파라미터

```
xgb_model = XGBRegressor(n_estimators=200, learning_rate=0.25, max_depth=8,
                          tree_method='hist', max_bin=32, n_jobs=4, random_state=156)
# XGBoost RMSLE: 0.50118
```

② HyperOpt 하이퍼파라미터 튜닝

전체 데이터(148만 건 × 10.6만 차원)로 매 시행마다 학습하면 탐색 시간이 지나치게 길어지므로, 학습 데이터에서 **20만 건을 샘플링**해 탐색용 학습/검증 세트를 구성하고 그 위에서 `tpe.suggest` 알고리즘으로 탐색했다. 목적 함수는 검증 셋 RMSLE(낮을수록 좋음)를 최소화한다.

파라미터	탐색 범위 / 분포	탐색 결과
<code>max_depth</code>	<code>quniform(4, 10, 1)</code>	9
<code>learning_rate</code>	<code>uniform(0.05, 0.5)</code>	0.3167
<code>n_estimators</code>	<code>quniform(100, 300, 20)</code>	260
<code>subsample</code>	<code>uniform(0.6, 1.0)</code>	0.9208
<code>colsample_bytree</code>	<code>uniform(0.6, 1.0)</code>	0.7506
<code>min_child_weight</code>	<code>quniform(1, 10, 1)</code>	5

탐색은 `max_evals=30`회 수행되었고 약 **1시간 53분**이 소요되었으며, 검증셋 기준 최저 loss는 0.5123이었다. 탐색된 최적 파라미터로 전체 데이터에 대해 XGBoost를 다시 학습한 결과는 다음과 같다.


```
best_xgb_model = XGBRegressor(max_depth=9, learning_rate=0.3167, n_estimators=260,
                              subsample=0.9208, colsample_bytree=0.7506, min_child_weight=5,
                              tree_method='hist', max_bin=32, n_jobs=4, random_state=156)
# HyperOpt 튜닝 XGBoost RMSLE: 0.48220
```

0.50118

XGBoost 기본

0.48220

XGBoost HyperOpt 튜닝

-0.019

튜닝에 의한 RMSLE 개선폭

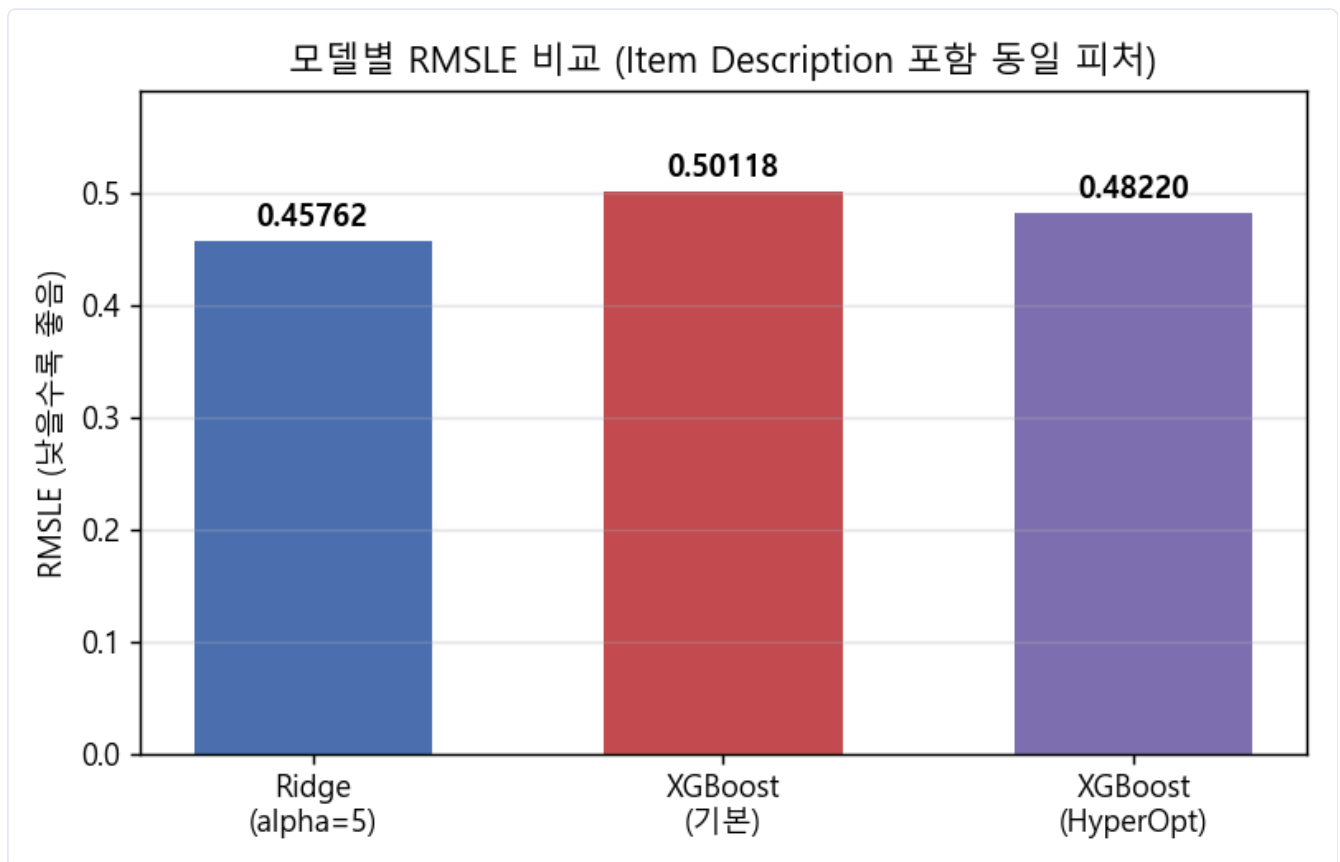
9.3 왜 트리 모델이 이 데이터에서는 불리한가

결정 트리는 “어떤 피처의 값이 얼마 이상/이하인가”를 기준으로 데이터를 반복해서 쪼갬다. 그런데 TF-IDF/원-핫으로 벡터화된 피처는 대부분의 값이 0이고 어쩌다 1이 되는 극단적으로 희소한 이진값에 가깝다. 이런 피처 10만 개 중에서 트리가 “어디를 기준으로 쪼갤지” 매번 탐색해야 하므로 연산량이 크게 늘고, 트리 하나가 한 번에 반영할 수 있는 분할 기준의 수는 제한적이라 개별 단어 하나하나의 미세한 신호를 촘촘히 담기 어렵다. 반면 릿지 같은 선형 모델은 피처 개수와 무관하게 “각 단어(피처)마다 가격에 미치는 영향(계수) 하나씩”을 동시에 계산하는 방식이라 이런 초고차원 희소 데이터에 훨씬 자연스럽게 들어맞는다. 이는 7절 아티클의 결론, 그리고 10절의 실측 결과와도 일치한다.

10 결과 및 결론

Ridge vs XGBoost(기본/튜닝) — 동일 피처(105,812차원) 비교

모델	설정	RMSLE
Ridge	alpha=5, item_description 포함	0.45762
XGBoost (기본)	lr=0.25, max_depth=8, hist	0.50118
XGBoost (HyperOpt 튜닝)	lr=0.317, max_depth=9, n_est=260 ...	0.48220



모델별 RMSLE 비교 (낮을수록 좋음)

해석

- 본 실험에서도 **Ridge가 튜닝된 XGBoost보다 RMSLE 기준으로 더 우수했다**(0.45762 vs 0.48220). 참고 아티클(Ridge 0.444 vs XGBoost 0.45)과 같은 방향이며, 초고차원 희소 텍스트 피처 환경에서 선형 모델의 우위가 재확인된다(9.3절 원인 분석).
- HyperOpt 튜닝으로 XGBoost는 0.50118 → 0.48220으로 약 0.019 개선되었으나 여전히 Ridge에 못 미친다. 30회 탐색에 약 1시간 53분이 소요된 것을 감안하면 **비용 대비 효과가 제한적**이다.
- 가격의 $\log_{1p}$ 변환(왜도 11.39 → 0.66, 5.2절)은 두 모델 모두에서 안정적인 학습에 필수적이었다. RMSLE가 로그 스케일 오차를 측정하므로, 타깃을 미리 로그 변환해 두는 것이 지표와 손실 함수를 일치시키는 합리적인 선택이었다.
- 결측치를 버리지 않고 'Other_Null' 별도 범주로 인코딩한 것(6.1절)은, `brand_name`의 42.7%에 달하는 결측이 실제로는 유의미한 정보("브랜드 없음")일 가능성이 높다는 5.1절의 관찰과 부합하는 처리였다.

10.1 파이프라인 요약



11 향후 계획

다음 단계로의 개선 방향

- **튜닝 탐색 확장:** HyperOpt 탐색을 20만 건 샘플이 아닌 전체 데이터 기반으로 수행하고, 탐색 횟수를 늘리며 `early_stopping_rounds`를 적용해 `n_estimators`를 데이터가 스스로 정하도록 한다. 10절에서 관찰된 “Ridge > 튜닝 XGBoost” 구도가 뒤집히는지 확인한다.
- **피처 엔지니어링 고도화:** 브랜드 × 카테고리 교차 피처, 가격 통계 파생 피처(단, 과적합 위험이 있어 검증 필요), 텍스트 길이·감성 지표 등을 추가 검토한다.
- **차원 축소:** TruncatedSVD 등으로 105,812차원 희소 행렬을 수백 차원으로 압축해 XGBoost의 학습 속도·메모리 효율을 개선하고, 축소된 밀집 표현에서 트리 모델의 성능이 개선되는지 재검증한다.